

WT Exp-6

Aim :- Interface the Arduino, connect the components together and run the project.

Code :-

```
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <SoftwareSerial.h>
#include <TinyGPS++.h>
#include <LiquidCrystal_I2C.h>

// LCD Configuration
LiquidCrystal_I2C lcd(0x27, 16, 2); // Adjust address if needed

// GPS Configuration
SoftwareSerial gpsSerial(3, 4); // RX=Pin 3, TX unused
TinyGPSPPlus gps;

// MPU6050 Configuration
Adafruit_MPU6050 mpu;
const float COLLISION_THRESHOLD = 2.0; // Adjust based on testing
bool collisionDetected = false; // Flag to track collision state
float collisionLat = 0.0;
float collisionLon = 0.0;
unsigned long lastDisplayChange = 0;
bool showLat = true; // Toggle between lat/long display

void setup() {
  Serial.begin(9600);
  gpsSerial.begin(9600);

  // Initialize LCD
  lcd.init();
  lcd.backlight();
  lcd.print("Initializing...");

  // Initialize MPU6050
  if (!mpu.begin()) {
    lcd.clear();
    lcd.print("MPU6050 Error!");
    while (true);
  }
```

```

    mpu.setAccelerometerRange(MPU6050_RANGE_16_G);
    delay(1000);
    lcd.clear();
}

void loop() {
    // Continuously read GPS data in background
    while (gpsSerial.available() > 0) {
        gps.encode(gpsSerial.read());
    }

    if (collisionDetected) {
        // Alternate between showing lat and long every 2 seconds
        if (millis() - lastDisplayChange > 2000) {
            lastDisplayChange = millis();
            showLat = !showLat;

            lcd.setCursor(0, 0);
            lcd.print("COLLISION! ");
            lcd.setCursor(0, 1);

            if (collisionLat != 0.0 && collisionLon != 0.0) {
                if (showLat) {
                    lcd.print("Lat:");
                    lcd.print(collisionLat, 6);
                    lcd.print(" ");
                } else {
                    lcd.print("Lon:");
                    lcd.print(collisionLon, 6);
                    lcd.print(" ");
                }
            } else {
                lcd.print("No GPS Fix ");
            }
        }
        return; // Stop further processing
    }

    // Read sensor data
    sensors_event_t a, g, temp;
    mpu.getEvent(&a, &g, &temp);

    // Calculate acceleration magnitude
    float accel = sqrt(sq(a.acceleration.x) + sq(a.acceleration.y) + sq(a.acceleration.z)) / 9.81;

    if (accel > COLLISION_THRESHOLD) {
        collisionDetected = true;
    }
}

```

```
lcd.clear();
lcd.print("Collision Detected!");
delay(2000); // Show detection message for 2 seconds

// Store current GPS data if available
if (gps.location.isValid()) {
    collisionLat = gps.location.lat();
    collisionLon = gps.location.lng();
}
return;
}

// Normal monitoring display
lcd.setCursor(0, 0);
lcd.print("Monitoring... ");
lcd.setCursor(0, 1);
lcd.print("Accel: ");
lcd.print(accel);
lcd.print("g ");

delay(100);
}
```

Connection:

Connecting the Arduino with all of its components :

Accelerometer/Gyroscope (MPU6050) to Arduino

The **MPU6050 sensor** detects sudden acceleration changes that indicate a collision. It uses the **I2C communication protocol**, requiring only **two pins** for data transmission.

MPU6050 Pin	Arduino Pin	Description
VCC	5V	Power Supply
GND	GND	Ground
SCL	A5	Serial Clock Line (SCL)
SDA	A4	Serial Data Line (SDA)

- ♦ **Tip:** The **MPU6050's default I2C address** is **0x68** or **0x69** (check using an I2C scanner).

GPS Module (NEO-6M) to Arduino

The GPS module retrieves the vehicle's location in case of a collision. It communicates using the UART (Serial Communication Protocol).

GPS Module Pin	Arduino Pin	Description
VCC	5V	Power Supply
GND	GND	Ground
TX (Transmit)	RX (Pin 4)	Sends GPS data to Arduino
RX (Receive)	TX (Pin 3)	Receives data from Arduino

LCD Display (16x2) to Arduino

The **LCD displays collision status** and **GPS coordinates**. It also uses **I2C communication** to minimize wiring.

LCD Display Pin	Arduino Pin	Description
SCL	A5	Serial Clock Line (SCL)
SDA	A4	Serial Data Line (SDA)
VCC	5V	Power Supply

GND

GND

Ground

- ♦ **Tip:** The **LCD I2C address is usually 0x27 or 0x3F**. Use an **I2C scanner** to confirm

```
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

void setup() {
  Serial.begin(115200); // Start the serial communication

  Wire.begin(); // Start the I2C communication
  mpu.initialize(); // Initialize the MPU-6050 sensor

  // Check if the sensor is connected properly
  if (mpu.testConnection()) {
    Serial.println("MPU6050 connected successfully");
  } else {
    Serial.println("MPU6050 connection failed");
  }
}

void loop() {
  // Read accelerometer and gyroscope data
  int16_t ax, ay, az;
  int16_t gx, gy, gz;

  mpu.getAcceleration(&ax, &ay, &az); // Get accelerometer values
  mpu.getRotation(&gx, &gy, &gz); // Get gyroscope values

  // Print values to serial monitor
  Serial.print("Accelerometer: ");
  Serial.print("X = "); Serial.print(ax);
  Serial.print(", Y = "); Serial.print(ay);
  Serial.print(", Z = "); Serial.println(az);

  Serial.print("Gyroscope: ");
  Serial.print("X = "); Serial.print(gx);
  Serial.print(", Y = "); Serial.print(gy);
  Serial.print(", Z = "); Serial.println(gz);
  delay(500); // Delay for half a second
}
```

Circuit Diagram :-



